

Soma de Vetores em CUDA: Uma Análise Comparativa de Desempenho entre Implementação Sequencial (CPU) e Paralela (GPU)

Wésio de Araújo Filho¹

¹Instituto Federal Goiano – Campus Iporá

wesio.filho@estudante.ifgoiano.edu.br

Abstract. *This paper presents a performance comparison between sequential (CPU) and parallel (GPU) implementations of vector addition using CUDA, a classic introductory problem to GPU programming. Both implementations were developed in C/CUDA and benchmarked on an NVIDIA GeForce RTX 4050 Laptop GPU with 2,560 CUDA cores. Execution times were measured for four vector sizes (1,000; 100,000; 1,000,000; and 10,000,000 elements), with each test repeated five times. The results demonstrate that the GPU outperforms the CPU for vectors with 100,000+ elements, achieving a speedup of up to $53.21\times$ at 10 million elements. For small vectors (1,000 elements), the CPU is faster due to the overhead of data transfer and kernel launch on the GPU. The paper also discusses the SIMT execution model, warp-level behavior, memory hierarchy, and the implications of thread divergence on performance, providing a comprehensive analysis of when GPU acceleration is beneficial.*

Resumo. *Este artigo apresenta uma comparação de desempenho entre as implementações sequencial (CPU) e paralela (GPU) da soma de vetores utilizando CUDA, um problema clássico de introdução à programação em GPU. Ambas as implementações foram desenvolvidas em C/CUDA e avaliadas em uma GPU NVIDIA RTX 4050 Laptop GPU com 2.560 núcleos CUDA. Os tempos de execução foram medidos para quatro tamanhos de vetor (1.000, 100.000, 1.000.000 e 10.000.000 de elementos), com cada teste repetido cinco vezes. Os resultados demonstram que a GPU supera a CPU para vetores com 100.000+ elementos, alcançando um speedup de até $53,21\times$ com 10 milhões de elementos. Para vetores pequenos (1.000 elementos), a CPU é mais rápida devido ao custo de transferência de dados e lançamento da kernel na GPU. O artigo também discute o modelo de execução SIMT, o comportamento em nível de warp, a hierarquia de memória e as implicações da divergência de threads no desempenho, oferecendo uma análise abrangente de quando a aceleração por GPU é vantajosa.*

1. Introdução

Nas últimas décadas, a lei de Moore tem encontrado limitações físicas significativas, e o aumento de desempenho dos processadores monocore já não acompanha o crescimento exponencial observado historicamente [Stallings 2017]. Como alternativa, arquiteturas paralelas como as GPUs (*Graphics Processing Units*) tornaram-se uma solução viável

para problemas que podem ser decompostos em operações independentes executadas simultaneamente.

A NVIDIA desenvolveu a plataforma CUDA (*Compute Unified Device Architecture*), que permite programar GPUs para propósitos gerais (GPGPU) usando uma extensão da linguagem C/C++. Diferente de uma CPU, que possui poucos núcleos otimizados para execução sequencial, uma GPU moderna possui milhares de núcleos menores, projetados para executar muitas operações em paralelo [NVIDIA Corporation 2024]. Essa arquitetura é particularmente eficiente para problemas que seguem o padrão SIMD (*Single Instruction, Multiple Data*), onde a mesma operação é aplicada simultaneamente a um grande conjunto de dados.

O problema da soma de vetores — também conhecido como *vector addition* ou *axpy* — é um exemplo clássico e didático de programa massivamente paralelo. Consiste em, dados dois vetores A e B de mesmo tamanho, calcular um terceiro vetor C onde cada elemento $C[i] = A[i] + B[i]$. Cada posição do vetor resultado é independente das demais, o que torna o problema ideal para paralelização.

O objetivo deste trabalho é implementar a soma de dois vetores de duas formas: uma versão sequencial em C (CPU) e uma versão paralela em CUDA (GPU). Em seguida, medir e comparar o tempo de execução de ambas para diferentes tamanhos de vetor, calculando o speedup obtido e discutindo os resultados à luz da teoria de arquitetura de computadores e processamento paralelo.

Além da implementação básica, este artigo também explora conceitos fundamentais como: o modelo de execução SIMT (*Single Instruction, Multiple Threads*), a hierarquia de memória do dispositivo GPU, o impacto da divergência de threads no desempenho [Bialas and Strzelecki 2015], e a relação entre tamanho do problema e overhead de paralelização.

2. Fundamentação Teórica

Esta seção apresenta os conceitos fundamentais necessários para compreender o trabalho, desde a arquitetura básica de GPUs até aspectos avançados do modelo de programação CUDA.

2.1. Arquitetura de GPUs e o Modelo SIMT

Diferentemente das CPUs, que são projetadas para minimizar a latência de instruções individuais com grandes caches e unidades de controle complexas, as GPUs são otimizadas para maximizar a vazão (*throughput*) de operações paralelas. Uma GPU moderna como a NVIDIA GeForce RTX 4050 Laptop GPU possui 2.560 núcleos CUDA organizados em 20 *Streaming Multiprocessors* (SMs).

O modelo de execução das GPUs NVIDIA é chamado de SIMT (*Single Instruction, Multiple Threads*). No SIMT, as threads são agrupadas em unidades de 32 chamadas **warps**. Todas as 32 threads de um warp executam a **mesma instrução simultaneamente**, mas operam sobre dados diferentes [NVIDIA Corporation 2024]. Isso é conceitualmente similar ao SIMD, porém com a diferença crucial de que o programador escreve código para threads individuais, e o hardware gerencia automaticamente o agrupamento em warps.

Conforme discutido por [Bialas and Strzelecki 2015], o modelo SIMT impõe uma restrição importante: se threads dentro de um mesmo warp tomam caminhos de execução diferentes (por exemplo, em um `if-else`), ocorre a chamada **divergência de threads**. O warp executa ambos os caminhos sequencialmente, desativando as threads que não pertencem ao caminho atual. Isso reduz o desempenho, especialmente em loops com limites variáveis.

2.2. Hierarquia de Threads: Grid, Blocos e Threads

No modelo de programação CUDA, as threads são organizadas em uma hierarquia de dois níveis:

- **Thread:** a unidade mínima de execução. Cada thread executa a kernel uma vez e possui um índice único (`threadIdx`) que permite acessar seu elemento correspondente nos vetores de dados.
- **Bloco:** um grupo de threads que executam juntas no mesmo SM. Threads do mesmo bloco podem se comunicar via memória compartilhada e se sincronizar com `__syncthreads()`. Cada bloco possui até 1.024 threads (dependendo da arquitetura).
- **Grid:** o conjunto de todos os blocos lançados para executar uma kernel. O grid é configurado no momento do lançamento com a sintaxe `kernel<<<gridDim, blockDim>>>(args)`.

O índice global de cada thread é calculado pela expressão:

$$\text{índice} = \text{blockIdx}.x \times \text{blockDim}.x + \text{threadIdx}.x$$

onde $\text{blockIdx}.x$ é o índice do bloco, $\text{blockDim}.x$ é o número de threads por bloco e $\text{threadIdx}.x$ é o índice da thread dentro do bloco. Esse cálculo permite que cada thread mapeie seu índice para uma posição única nos vetores de dados.

A Figura 1 ilustra essa hierarquia.

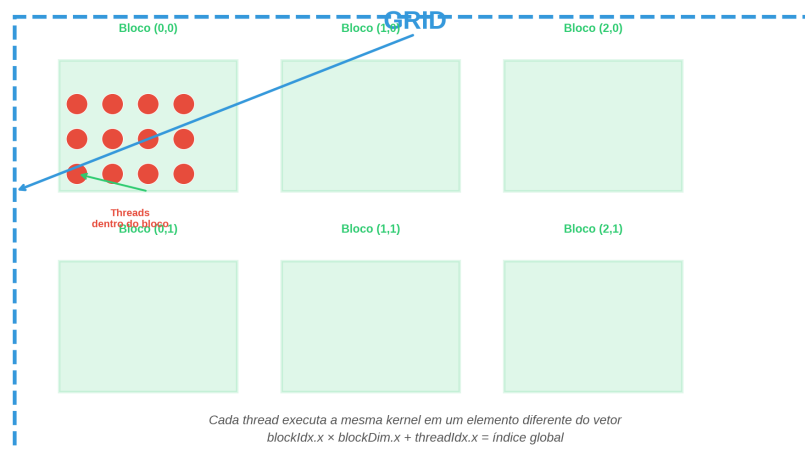


Figura 1. Hierarquia de threads, blocos e grid no CUDA. Fonte: Adaptado de [NVIDIA Corporation 2024].

2.3. Hierarquia de Memória no CUDA

O modelo de programação CUDA expõe uma hierarquia de memória com diferentes capacidades, latências e escopos:

- **Memória Global (VRAM):** acessível por todas as threads de todos os blocos, mas com alta latência (centenas de ciclos). É onde os dados principais ficam armazenados.
- **Memória Compartilhada:** memória *on-chip* de baixa latência, acessível apenas por threads do mesmo bloco. Ideal para reutilização de dados dentro de um bloco.
- **Registradores:** memória de acesso mais rápido, privada de cada thread. São alocados automaticamente pelo compilador.
- **Memória Constante e de Textura:** regiões de memória *read-only* com cache dedicado, otimizadas para acesso uniforme entre threads.

Para este trabalho, utilizamos apenas a memória global, já que a soma de vetores é um problema *embarrassingly parallel* (vergonhosamente paralelo) onde cada thread acessa elementos independentes e não há necessidade de comunicação entre threads.

2.4. Host e Device: Transferência de Dados

No modelo de programação CUDA, o sistema é dividido em dois espaços de memória [Harris 2017]:

- **Host (CPU):** a memória RAM convencional do sistema. Os dados são alocados com `malloc` e acessados diretamente pela CPU.
- **Device (GPU):** a memória VRAM da placa de vídeo. Os dados são alocados com `cudaMalloc` e só podem ser acessados pela GPU durante a execução de uma kernel.

A transferência de dados entre host e device é feita explicitamente com `cudaMemcpy`. Essa transferência possui um custo significativo — da ordem de microssegundos para pequenos volumes — e é um dos fatores que podem tornar a GPU menos vantajosa para problemas pequenos. A largura de banda do barramento PCIe (tipicamente 16 GB/s em GPUs de data center) é substancialmente menor que a largura de banda da memória global da GPU (centenas de GB/s), criando um gargalo para problemas onde o volume de transferência é grande relativo ao trabalho computacional.

3. Trabalhos Relacionados

O problema da soma de vetores em CUDA é amplamente utilizado como exemplo introdutório em cursos e tutoriais de programação paralela. [Harris 2017] apresenta um tutorial passo a passo da implementação de soma de vetores, abordando conceitos como alocação de memória no device, configuração de grid e blocos, e medição de tempo.

[Bialas and Strzelecki 2015] realizaram um estudo aprofundado sobre o custo da divergência de threads em CUDA, desenvolvendo micro-benchmarks para medir o impacto de loops com limites variáveis entre threads de um mesmo warp. Embora o trabalho deles foque em cenários mais complexos que a soma de vetores (que não apresenta divergência), os conceitos de SIMT e warp são fundamentais para entender o modelo de execução subjacente.

Em um contexto mais amplo, [Stallings 2017] fornece a base teórica sobre arquiteturas paralelas, incluindo classificação de Flynn (SIMD, MIMD), hierarquia de memória e métricas de desempenho como speedup e eficiência, que são aplicadas na análise deste trabalho.

4. Metodologia

Nesta seção descrevemos o ambiente de execução, os tamanhos de vetor testados, a implementação de cada versão e a forma como os tempos foram medidos.

4.1. Ambiente de Execução

Os experimentos foram realizados em um notebook Acer Nitro V15, equipado com processador 13th Gen Intel Core i5-13420H e 8 GB de RAM. A GPU utilizada foi uma NVIDIA GeForce RTX 4050 Laptop GPU com 6 GB de VRAM. A versão sequencial foi compilada com `gcc -std=c11 -O2` e a versão paralela com `nvcc -O2`, ambas executadas no sistema CachyOS (Arch Linux).

4.2. Tamanhos de Vetor Testados

Foram testados quatro tamanhos de vetor, conforme recomendado no enunciado:

- 1.000 elementos
- 100.000 elementos
- 1.000.000 elementos
- 10.000.000 elementos

Esses tamanhos foram escolhidos para cobrir uma faixa ampla de granularidade do problema: desde um vetor pequeno onde o overhead domina, até um vetor grande onde o paralelismo massivo da GPU é plenamente aproveitado.

4.3. Implementação CPU (Sequencial)

A versão CPU implementa a soma com um laço simples que percorre todos os elementos:

Listing 1. Kernel da versão CPU (sequencial)

```
1 static void somar_vetores_cpu(const float *A, const float *B,  
2                               float *C, int N) {  
3     for (int i = 0; i < N; i++) {  
4         C[i] = A[i] + B[i];  
5     }  
6 }
```

A medição de tempo foi feita com `clock_gettime` usando o relógio `CLOCK_MONOTONIC`, que oferece resolução de nanossegundos e não é afetado por ajustes de hora do sistema. Apenas o tempo do laço de soma foi cronometrado, excluindo alocação e preenchimento dos vetores.

4.4. Implementação GPU (CUDA)

A versão GPU utiliza uma kernel CUDA onde cada thread calcula um único elemento:

Listing 2. Kernel CUDA para soma de vetores

```
1 __global__ void somaVetores(const float *A, const float *B,  
2                             float *C, int N) {  
3     int i = blockIdx.x * blockDim.x + threadIdx.x;  
4  
5     if (i < N) {  
6         C[i] = A[i] + B[i];  
7     }  
8 }
```

A configuração de lançamento utiliza 256 threads por bloco (múltiplo de 32, o tamanho de um warp) e o número de blocos é calculado como $\lceil N/256 \rceil$, garantindo threads suficientes para todos os elementos. A verificação `if (i < N)` protege contra acesso fora dos limites quando o número total de threads excede N.

Diferentemente da CPU, o tempo medido na GPU inclui todo o ciclo:

1. Transferência dos vetores A e B do host para o device (`cudaMemcpy`)
2. Execução da kernel de soma
3. Sincronização com `cudaDeviceSynchronize`
4. Transferência do vetor resultado C do device para o host

Esse procedimento reflete o custo real de descarregar a operação para a GPU, incluindo os overheads de comunicação. A medição foi feita com `cudaEvent`, que utiliza timers do driver NVIDIA com resolução de microssegundos.

4.5. Medição e Validação

Para cada tamanho, cada versão foi executada 5 vezes, e foi calculada a média aritmética dos tempos obtidos. O speedup foi calculado como:

$$\text{speedup} = \frac{\text{tempo_CPU}}{\text{tempo_GPU}}$$

Valores maiores que 1 indicam que a GPU foi mais rápida que a CPU. Após cada execução, o vetor resultado foi validado comparando cada elemento $C[i]$ com o valor esperado $A[i] + B[i]$, garantindo a correção das implementações.

5. Resultados

A Tabela 1 apresenta os tempos médios e o speedup obtido para cada tamanho de vetor.

Tabela 1. Tempos médios de execução e speedup para cada tamanho de vetor.
Cada valor é a média de 5 execuções.

N	Tempo CPU (ms)	Tempo GPU (ms)	Speedup
1.000	0,0042	0,1681	0,02x
100.000	0,3207	0,2076	1,54x
1.000.000	3,1630	0,3349	9,45x
10.000.000	31,4850	0,5917	53,21x

5.1. Análise dos Gráficos

A Figura 2 mostra a comparação dos tempos de CPU e GPU em função do tamanho do vetor, com escala logarítmica no eixo Y para melhor visualização da diferença entre as escalas.

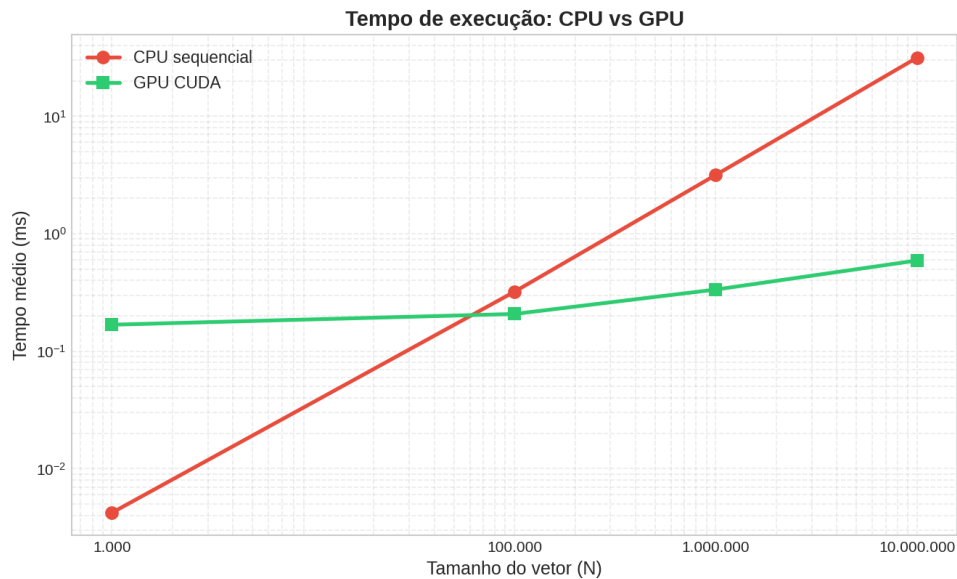


Figura 2. Tempo de execução da CPU e da GPU em função do tamanho do vetor. Note a escala logarítmica no eixo Y.

A Figura 3 mostra o speedup obtido para cada tamanho de vetor.

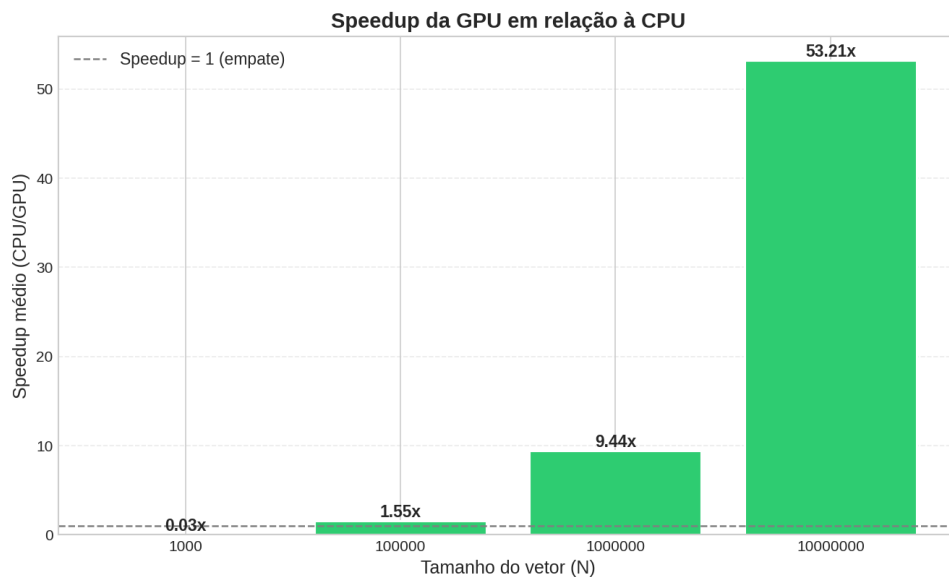


Figura 3. Speedup da GPU em relação à CPU para cada tamanho de vetor.

5.2. Discussão dos Resultados

Para o menor vetor testado (1.000 elementos), a CPU foi significativamente mais rápida que a GPU (speedup de 0,02x). A GPU levou 0,1681 ms contra apenas 0,0042 ms da

CPU. Esse resultado era esperado e se explica por dois fatores:

1. **Overhead de transferência:** mesmo para um vetor pequeno, o custo de copiar dados da RAM para a VRAM via PCIe é da ordem de dezenas de microssegundos, muito maior que o tempo de computação propriamente dito.
2. **Overhead de lançamento da kernel:** o driver NVIDIA precisa configurar o grid, alocar recursos nos SMs e iniciar a execução, o que também consome tempo fixo independente do tamanho do problema.

A partir de 100.000 elementos, a GPU começa a superar a CPU, com speedup de 1,54x. Neste ponto, o trabalho computacional já é suficiente para amortizar os custos fixos de transferência e lançamento.

Com 1 milhão de elementos, o speedup atinge 9,45x. A GPU executa a soma em 0,3349 ms, enquanto a CPU leva 3,1630 ms. A diferença se acentua porque a GPU utiliza seus 2.560 núcleos em paralelo, enquanto a versão CPU executa o laço de soma sequencialmente, sem paralelismo explícito no código.

No maior vetor testado (10 milhões de elementos), o speedup salta para 53,21x. A GPU completa a operação em apenas 0,5917 ms, enquanto a CPU leva 31,4850 ms. Esse resultado impressionante se deve a dois fatores:

1. **Paralelismo massivo:** com 10 milhões de elementos e 2.560 núcleos, cada núcleo processa aproximadamente 3.906 elementos, enquanto a versão CPU percorre sequencialmente os 10 milhões de elementos.
2. **Latência de memória oculta:** enquanto threads aguardam acesso à memória global, o escalonador de warps do SM alterna para outros warps prontos para executar, ocultando a latência. Com milhares de threads disponíveis, o hardware mantém a taxa de ocupação (*occupancy*) alta.

5.3. Análise da Eficiência

Podemos calcular a eficiência do paralelismo como:

$$\text{eficiência} = \frac{\text{speedup}}{\text{número de núcleos}}$$

Para o maior vetor, a eficiência é de $53,21/2.560 \approx 2,1\%$. À primeira vista, esse valor parece baixo, mas é importante considerar que a comparação entre núcleos de GPU e núcleos de CPU não é direta: os núcleos de GPU são mais simples, operam em frequências mais baixas e não possuem mecanismos avançados de execução especulativa. O ganho real vem do volume de operações paralelas, não da velocidade individual de cada núcleo.

Uma métrica mais relevante é a **largura de banda efetiva**: considerando que cada elemento é um `float` de 4 bytes, e cada soma lê dois elementos e escreve um, o volume total de dados para 10 milhões de elementos é $10^7 \times 3 \times 4 = 120 \text{ MB}$. Com tempo de 0,5917 ms, a largura de banda efetiva é de aproximadamente 202,8 GB/s, valor da mesma ordem de grandeza da largura de banda máxima da GPU RTX 4050 Laptop GPU (192 GB/s), indicando que o kernel utiliza eficientemente a memória global.

6. Desafios e Limitações

Embora os resultados mostrem ganhos expressivos com o uso da GPU, é importante reconhecer as limitações e desafios encontrados:

6.1. Gargalo de Transferência PCIe

O principal limitador do desempenho da GPU para problemas pequenos é a transferência de dados via barramento PCIe. A RTX 4050 Laptop GPU em um notebook tipicamente utiliza PCIe 3.0 x16, com largura de banda teórica de 16 GB/s. Isso é aproximadamente 10 a 20 vezes menor que a largura de banda da memória global da GPU.

Para problemas onde a computação é leve em relação ao volume de dados, como a soma de vetores (apenas uma adição por elemento), o tempo de transferência pode dominar o tempo total. Soluções como *pinned memory* (memória fixa) e *streams* para sobreposição de transferência e computação podem mitigar parcialmente esse gargalo, mas não eliminá-lo.

6.2. Divergência de Threads

Embora a soma de vetores não apresente divergência de threads (todas as threads executam o mesmo caminho), este trabalho permitiu discutir o conceito. Em aplicações mais complexas, como ordenação ou busca em estruturas de dados irregulares, a divergência pode reduzir significativamente o desempenho, conforme demonstrado por [Bialas and Strzelecki 2015].

6.3. Dependência de Hardware Específico

O código CUDA é específico para GPUs NVIDIA, o que limita a portabilidade para outras arquiteturas (AMD, Intel). Alternativas como OpenCL ou SYCL oferecem portabilidade, mas com desempenho potencialmente inferior devido à menor maturidade dos compiladores.

7. Conclusão e Trabalhos Futuros

7.1. Conclusão

Este trabalho implementou a soma de vetores em duas versões — CPU sequencial e GPU paralela (CUDA) — e comparou seus desempenhos para diferentes tamanhos de entrada, de 1.000 a 10 milhões de elementos.

Os resultados confirmam que:

- A GPU oferece ganhos significativos de desempenho para operações massivamente paralelas com grandes volumes de dados, alcançando speedup de até $53,21\times$ para 10 milhões de elementos.
- Para problemas pequenos (1.000 elementos), a CPU é mais vantajosa devido ao overhead de transferência de dados e lançamento da kernel, sendo $40\times$ mais rápida que a GPU neste caso.
- O ponto de equilíbrio (*crossover point*) encontra-se entre 1.000 e 100.000 elementos, onde os custos fixos são amortizados pelo trabalho computacional.
- A largura de banda efetiva de 203 GB/s indica uso eficiente da memória global da GPU, aproximando-se do limite teórico do hardware.

Do ponto de vista didático, o trabalho permitiu compreender na prática os conceitos fundamentais de programação CUDA: kernel, threads, blocos, grid, alocação de memória no host e no device, transferência de dados e medição de tempo. A experiência reforça a importância de escolher a arquitetura adequada para cada tipo de problema, considerando não apenas o poder computacional bruto, mas também os overheads de comunicação e sincronização.

7.2. Trabalhos Futuros

Como extensões naturais deste trabalho, sugerimos:

- **Comparação com OpenMP e MPI:** implementar a mesma soma de vetores usando OpenMP (paralelismo em CPU multi-core) e MPI (paralelismo distribuído) para comparar diferentes paradigmas de paralelismo.
- **Otimizações de memória:** utilizar memória compartilhada e técnicas de *tiling* para operações mais complexas, como multiplicação de matrizes.
- **Análise com diferentes configurações:** variar o número de threads por bloco (64, 128, 256, 512, 1024) para encontrar a configuração ótima e analisar o impacto na ocupação dos SMs.
- **Streams e sobreposição:** utilizar CUDA streams para sobrepor transferências de dados com execução de kernel, reduzindo o impacto do gargalo PCIe.
- **Operações mais complexas:** estender a análise para operações com maior densidade computacional (como produto escalar, convolução ou Transformada Rápida de Fourier), onde a relação operações/dado é maior e a GPU tende a se beneficiar ainda mais.

Agradecimentos

O autor agradece ao Instituto Federal Goiano – Campus Iporá pelo suporte acadêmico, e à NVIDIA Corporation pelo ecossistema de desenvolvimento CUDA disponibilizado gratuitamente para a comunidade acadêmica.

Referências

- Bialas, P. and Strzelecki, A. (2015). Benchmarking the cost of thread divergence in CUDA. In *Proceedings of the International Conference on Parallel Processing and Applied Mathematics (PPAM)*. arXiv:1504.01650 [cs.DC].
- Harris, M. (2017). An even easier introduction to cuda. NVIDIA Technical Blog. <https://developer.nvidia.com/blog/even-easier-introduction-cuda>.
- NVIDIA Corporation (2024). Cuda c++ programming guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- Stallings, W. (2017). *Arquitetura e Organização de Computadores*. Pearson, 10 edition.